

School of Computer Science and Engineering, UNSW



Never Stand Still

WK04-02: Blockchain Taxonomy

Software Architecture for Blockchain Applications,
COMP6452

Salil Kanhere, Nadeem Ahmed

Agenda

Taxonomy definition

Blockchain taxonomy

- (De)centralization
- Consensus protocols
- Ledger structures
- Mining & ledger distributions

- First, we'll discuss what a taxonomy is.
- Then we discuss a set of classifications like decentralisation, consensus protocols, auxiliary blockchains that supplement an existing blockchain, and different ledger structures. The textbook includes a few other classifications.

Taxonomy

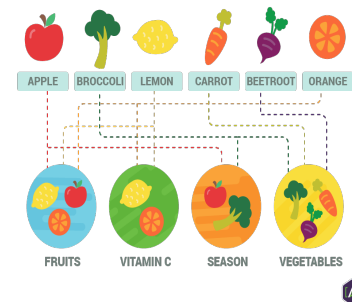
Practice & science of classification of things or concepts, including principles that underlie such classification

Diverse range of blockchain has emerged

- Complex internal structures, many variants, & configurations
- Difficult to compare & contrast

Blockchain taxonomy

- Dimensions & categories for classifying blockchains
- Enhance understanding by
 - Exploring conceptual design space
 - Compare & evaluate design options
 - Assess their impact on quality attributes



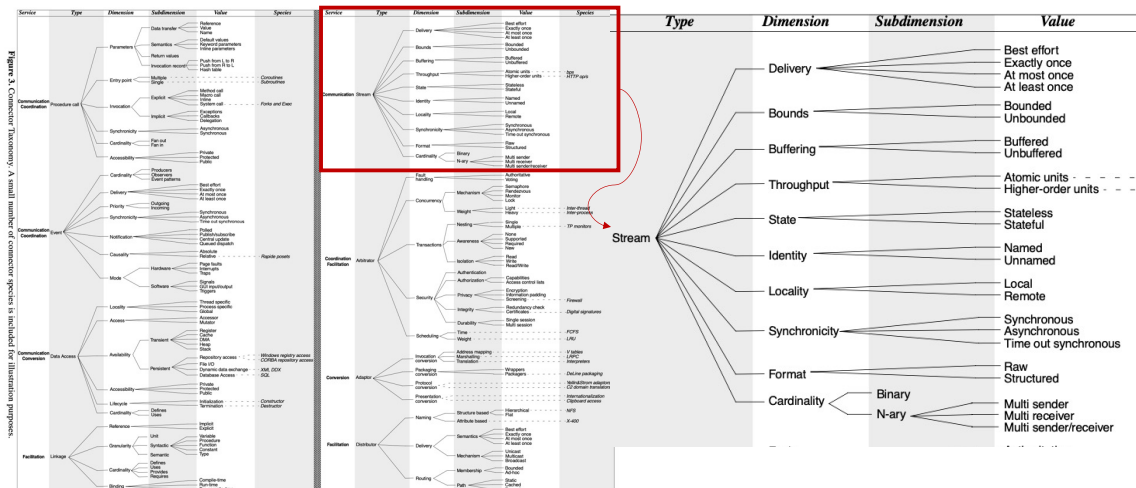
WK 04-02

3



- Taxonomy is the practice and science of classification of things or concepts, including the principles that underlie such classification.
 - Here we see different classifications as fruits and vegetables. They are also classified based on whether they contain vitamin C and whether they are seasonal (growing or harvest) fruits and vegetables.
 - Fruit and Vegetable can be classified based on botanical characteristics. Fruit is a seed-bearing structure that develops from the ovary of a flowering plant. Vegetables are all other plant parts, like roots, leaves, and stems.
- While taxonomies are used heavily in biology and zoology, they have also been used in software architecture to understand existing technologies.
- A diverse range of blockchains has emerged without much detail on their internal designs, product data, or performance. Given their complex internal structures, many variants, and configurations it is difficult to compare and contrast them.
- A taxonomy on blockchains defines dimensions and categories for classifying blockchains and ways of using them in systems.
- The presented taxonomy captures major architecturally-relevant characteristics of various blockchains and indicates their support for various quality attributes. It was informed by existing industrial products, technical forums, academic literature, and our own experience of using blockchains and developing prototypes.
- Such a taxonomy enhances understanding by:
 - Exploring conceptual design space as building application on blockchain need to consider features and configuration.
 - Comparing and evaluate design options.
 - It can help consider impact of different design decision on quality attributes for the overall system.

Software Connector Taxonomy



Source: Mehta et al., "Towards a taxonomy of software connectors", In Proc. 22nd Intl. Conf. on Software Engineering, June 2000.

WK 04-02

4



- Here's a taxonomy from a paper that presents a comprehensive classification framework and taxonomy of software connectors.
 - Software connector is the fundamental building block of software architecture that deals with the interaction between software components.
- The taxonomy proposed in the paper can be used both to understand existing software connectors and to propose new software connectors that can fill the gaps within the state of the art.
- The diagram on the left shows the overall view of the software connector taxonomy. Organized in a tree structure.
- The diagram on the right zooms into the sub-tree about the stream software connector. As we can see that the taxonomy uses 10 different dimensions to further classify the stream software connectors.
 - For example, in terms of delivery, there are 4 values representing the connector uses best effort strategy, the connector delivers exactly once or delivers at most once or at least once.
 - In terms of buffering, some connectors buffered the data packages, some don't. And in terms of state, some connector is stateless, while others need to maintain the state.
- Such taxonomy proposed in the paper can be used both to understand existing software connectors, facilitate the selection of existing software connectors, and propose new software connectors that can fill the gaps within the state-of-art, for example, by exploring different combinations of the possible values of different dimensions.

Agenda

Taxonomy definition

Blockchain taxonomy

- (De)centralization
- Consensus protocols
- Ledger structures
- Mining & ledger distributions

Taxonomy: A Glimpse - Classification

	Permissionless	Permissioned
Public	Consensus: Proof-of-X Permission management - Blockchain layer - Application layer (optional) Incentive: Blockchain layer	Consensus - Proof-of-X - PBFT, Federated consensus, Round Robin, etc. Permission management - Blockchain layer - Application layer (optional) Incentive: - Blockchain layer - Governance around permissions
Private	Consensus - Proof-of-X - PBFT, Federated consensus, Round Robin, etc. Permission management: - Blockchain layer - Network layer - Application layer (optional) Incentive: Governance around access	Consensus - Proof-of-X - PBFT, Federated consensus, Round Robin, etc. Permission management: - Blockchain layer - Network layer - Application layer (optional) Incentive: Governance around access permissions

WK 04-02

6



- The following 2 slides give a much-simplified classification, which is represented by a 2 by 2 matrix.
- 4 categories based on 2 dimensions: public/private and permissioned/permissionless
- Compared based on 3 aspects, including consensus protocol, permission management, and incentives.
- Permissionless-public (Bitcoin & Ethereum).
 - These blockchains are the most widely known, where everyone can join the blockchain network to submit and validate transactions without permission.
- Permissioned public (Ripple and Avalanche)
 - These blockchains are public but need permissions to validate transactions and build blocks. However, the information stored on such a blockchain is publicly accessible to all the participants.
- Permissionless private (Ethereum on a private network)
 - Is a rare option, but is still technically possible, where the blockchain is deployed within a private network environment. People within that network environment don't need extra permission to join that blockchain network.
- Permissioned private (Hyperledger Fabric and R3 Corda)
 - These blockchains are also widely used in business, where the blockchain is deployed within a private network environment. Organizations need to get permission to join the network. A private-permissioned blockchain can better protect data privacy and confidentiality because the blockchain is not deployed publicly, accessible to all that have internet access.
- In terms of consensus protocol, the permissionless blockchain normally use some form of Proof-of-X consensus protocol, like proof-of-work used by Bitcoin and proof-of-stake in Ethereum 2.0.
- The other 3 categories can use both proof-of-X and PBFT Byzantine practical fault-tolerant protocol, federated consensus, or round robin.
- Permission management in private networks usually involves a firewall.
- Incentive in permissionless public: transaction fee and block reward.
- Private blockchains, including both permissionless and permissioned private blockchains, rely on governance around access permissions.

Taxonomy : A Glimpse – Quality Tradeoffs

	Permissionless	Permissioned
Public	Immutability +++ (#Nodes, Consensus, Topology)	Immutability ++
	Integrity +++ (#Nodes, Consensus, Topology)	Integrity ++
	Transparency ++ (Access control)	Transparency ++
	Availability +++ (#Nodes, Topology)	Availability ++
	Performance + (Consensus, latency)	Performance ++
	Cost Efficiency +	Cost Efficiency ++
Private	Immutability +	Immutability +
	Integrity +	Integrity +
	Transparency +	Transparency +
	Availability +	Availability +
	Performance +++	Performance +++
	Cost Efficiency +++	Cost Efficiency +++

WK 04-02

7



- Matrix shows the impact of these 4 categories of blockchain on quality attributes.
- Number of pluses represents how well the quality is compared with other alternatives.
- Aspects in brackets are the aspects of the blockchain that affect the quality.
- Among the 4 categories, permissionless public blockchain has the best immutability, but the worst performance and cost efficiency.
- Immutability and integrity are good due to the size of the network. The larger size of the network, the more computational power provided by the whole network and the more difficult for a single party to attack the blockchain network or change the data stored on the blockchain.
- The consensus protocol used by a blockchain can also affect the fundamental properties provided by a blockchain. For example, proof-of-work can provide the best immutability in general since it is nearly impossible for a single participant within the blockchain network to host enough computational power to control the whole blockchain network.
- Transparency is affected by access control.
- Availability is mainly affected by the number of nodes within the blockchain network.
- Performance is worse due to latency caused by consensus on a large network.
- Cost is worse due to the cost associated with the maintenance and management of a large network.

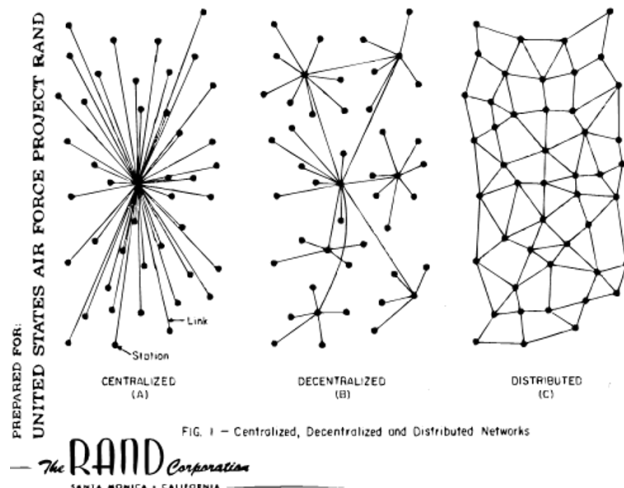
Agenda

Taxonomy definition

Blockchain taxonomy

- (De)centralization
- Consensus protocols
- Ledger structures
- Mining & ledger distributions

Decentralisation



Centralised systems won't survive a targeted attack

- Even with reliable hardware, software, & networks

A distributed configuration can achieve highly reliable systems

- With unreliable hardware, software, & networks

Blockchains/DLTs are either decentralised or distributed

- This is a diagram from Paul Baran, "On Distributed Communications Networks", IEEE Transactions on Communications Systems, Vo. 12, no. 1, March 1964, which proposed a distributed system for the management of missile networks in the USA.
- It considered survivability in the cases of enemy attacks directed against nodes, network links, or combinations of nodes and links.
- Figure A and B are the telecommunication provider approach at that time. Even though such designs include reliable hardware, software, and networks they won't survive targeted attacks.
- Paul Baran proposed the idea that a distributed configuration can achieve highly reliable systems even with unreliable hardware, software, and networks.
- This has led to the development of a network to manage the missile system in the USA, which eventually became the Internet.
- Some blockchain designs are decentralised (most private-permissioned blockchains) while others are distributed (most public blockchains).

ISO 22739:2020 Terminology – Distributed vs Decentralised

Distributed system

- "System in which components located on networked computers communicate & coordinate their actions by interacting with each other"
- Terminology covers decentralised & distributed designs in RAND project figure

Decentralised system

- "Distributed system wherein **control** is distributed among persons or organisations participating in the operation of system"
- "Note 1 to entry: In a decentralised system, distribution of control among persons or organisations participating in the system is determined by its design"
- Decentralised systems are distributed systems with distributed administrative control

- ISO 22739:2020 Blockchain and Distributed Ledger Technologies standard's terminology discusses the distributed and decentralised slightly differently (this may not be consistent with the textbook which came out before ISO definition).
- Distributed systems here cover decentralised and distributed designs in the RAND project figure in the previous slide.
- ISO takes the view that decentralised systems are distributed systems with distributed administrative control. Whereas many distributed systems can be under a single administration.

Centralisation vs Decentralisation

Design Decision	Option	Impact			
		Fundamental properties	Cost efficiency	Performance	#Failure points
Fully Centralised	Services with a single provider (<i>e.g.</i> , governments, courts)	⊕	⊕⊕⊕	⊕⊕⊕	1
	Services with alternative providers (<i>e.g.</i> , banking, online payments, cloud services)				
Partially Centralised & Partially Decentralised	Permissioned blockchain with permissions for fine-grained operations on the transaction level (<i>e.g.</i> , permission to create assets)	⊕⊕	⊕⊕	⊕⊕	*
	Permissioned blockchain with permissioned miners (write), but permission-less normal nodes (read)				
Fully Decentralised	Permission-less blockchain	⊕⊕⊕	⊕	⊕	Majority (nodes, power, stake)

WK 04-02

11



- Fundamental Properties: Immutability, non-repudiation, integrity, transparency and equal rights
- Decentralisation is one of the key design aspects of blockchain technology. It devolves responsibility and capability from a central location or authority.
- In a centralised system, all users rely on a central authority to mediate transactions. For e.g., in a bank, customers rely on the bank's systems to correctly adjust their account balances after a bank transfer.
- Manipulate the whole system: updating backend databases, or upgrading the software happen under a single administrative entity
- This leads to a single point of failure for the whole system → minimised through geographical replication.
- A fully decentralised system allows people to reach an agreement on who owns what without having to trust each other or a separate third party.
- Such a design is highly available, for e.g., as every full node in Bitcoin has a full copy of the blockchain and processes every transaction.
- There is a spectrum of possibilities between centralisation and decentralisation.
- Ripple – has a centrally selected 100 nodes. Monax – claims to be the 1st permissioned blockchain.
- Hyperledger Fabric is not quite fully decentralized. Kafka-based ordering service (before ver 2.0) for a crash fault-tolerant ordering service is not fully decentralized. Raft-based ordering service introduced in ver 1.4 gets closer to having a decentralized network. The eventual plan is to have a fully decentralized Fabric network with the BFT (byzantine fault tolerance) ordering service

Full Centralisation

Centralised data & administrative control

Services with a single provider

- E.g., governments, courts, business monopolies
- Single point of failure

Services with alternative providers

- E.g., telcos, banking, online payments, cloud services
- Failure of a single service provider or business failure affects its users
 - E.g., When a mobile payment company goes bankrupt & switches off the servers, all customers are affected

- Centralised data and administratively centralised control. Data may be physically distributed, but still under a single authority's control.
- Monopoly service providers, including governments and courts within a jurisdiction, and business monopolies.
 - Single point of failure for its users.
 - Failure of a single service provider only affects its users.
- Competing alternative providers, such as banks, online payments, or cloud computing providers.
 - Users may switch providers or may be able to use multiple providers simultaneously.
 - However, for business failures, e.g., when a company behind IoT devices goes bankrupt and switches off the servers, all devices stop working.
 - So the failure could be technical or business-related.

Full Decentralisation

Decentralised data & administrative control

Permissionless public blockchains

- Most blockchains logically centralises data

Completely open

- New users can join, validate TX, or mine block at any time
- Could be across jurisdictional boundaries undermining regulatory controls

Need protection against Sybil attacks

- PoW – Total amount of computational power rather than number of nodes is important for integrity
- 51% attack possible by accumulating massive computing power, but very costly

- In these systems both data and administration are decentralised.
- Permissionless public blockchains are completely open: new users can at any time join the network, validate transactions, and mine blocks.
 - While Bitcoin and Ethereum come under this classification they logically centralise data as every node has the same data. Ethereum 2.0 is trying to also decentralise data with sharding.
- Decentralised systems use anonymous validators to protect against *Sybil attacks*, where attackers create many hostile anonymous nodes.
 - A Sybil attack is an attack on a computer network service in which an attacker subverts the service's reputation system by creating many pseudonymous identities and uses them to gain a disproportionately large influence.
 - Bitcoin's proof-of-work (PoW) mechanism focuses is not the total number of nodes that is important for integrity but rather on the total amount of computational power.
 - A decentralised system can be defeated if there is a majority of authority (nodes, computational power, or stakeholding). For e.g., in 51% attack, the attacker accumulates more computing power (or stake) than the rest of the network
 - Game-theoretic attacks can change this threshold, requiring a higher (e.g. 66%) majority to maintain integrity.

Partial (De)centralisation

Partially centralised data & administrative control

Permissioned blockchains require authorities to act as a gatekeeper

- Permission to join the network (read)
- Permission to initiate TX (write)
- Permission to mine

Permissioned blockchain with permissions for fine-grained operations

- Permission to create assets

Permissioned blockchain with permissioned miners (write), & permission-less nodes (read)

- New users can join & validate TXs, but can't mine blocks

- These systems partially centralised data and administrative control.
- A blockchain may be permissioned in requiring that one or more authorities act as a gate for participation.
 - The permissions may include the ability to read ledger data, write data into the ledger, and build blocks.
 - Based on this, some designs allow reads to anyone while writing only to a selected set of users. Others require permission to read, write, and mine (i.e., build blocks).
- The code for public blockchains can also be deployed on private networks to create a kind of permissioned blockchain using network access controls.
- Permission information can be stored either on-chain or off-chain.

Partial (De)centralization Cont.

More suitable in regulated industries

- Control on who can mine
 - E.g., Know-Your-Customer (KYC) regulations require establishing real-world identity of transacting parties
- Knowing a party establishes some level of trust
 - E.g., overcoming Sybil attacks
 - At least, legal system as backstop
- Enhanced trust reduces need for extreme consensus protocols & large number of miners
 - High performance & efficient blockchains/DLTs
- Permissioned nodes may still get attacked leading to misbehaviour
 - Still need some number of nodes. Question is how many?
 - E.g., Byzantine Fault Tolerance (BFT) based protocols use $3f + 1$ nodes to guard against f failures

- Because of the permissions, decentralised networks are more suitable for regulated industries.
- There's control over who can read, write, and mine. Mining is perhaps the most important as they determine what persists on the ledger.
- When you have control over who can mine, like KYC in banking, it establishes some level of trust.
 - For e.g., at least Sybil attacks can be avoided because a party can't create so many pseudo accounts/nodes to get disproportionate control of the system.
 - Even if such technical solutions are limited or failed, knowing a party we can rely on the legal system as a backstop.
- Through this enhanced trust, we can avoid the need to have extreme consensus protocols like PoW with Nakamoto consensus and reduce the number of miners needed to prevent attacks that may accumulate/combine many nodes.
 - Relatively small networks with efficient consensus protocols lead to high-performance and efficient networks.
- However, even the permissioned nodes may misbehave, e.g., the node may be attacked or run by a crooked owner. So, you would still need some number of nodes. However, it's not straightforward to answer this question.
 - For e.g., Byzantine Fault Tolerance (BFT) based protocols use $3f + 1$ nodes to guard against f failures.

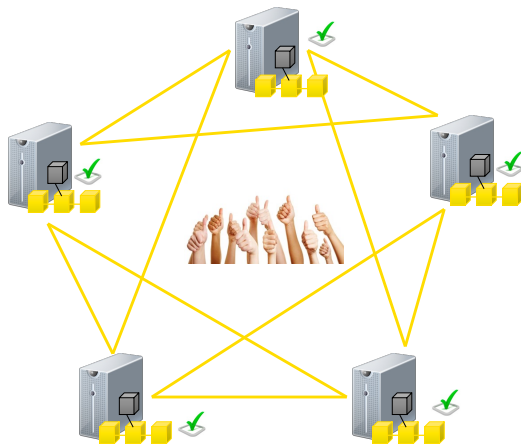
Agenda

Taxonomy definition

Blockchain taxonomy

- (De)centralization
- **Consensus protocols**
- Ledger structures
- Mining & ledger distributions

Consensus Protocol



Miners generate new blocks & propagate to peers in the blockchain network

Miners compete for new blocks

- Different sets of TXs in a block
- Multiple solutions to PoW puzzle
- Network delays

Miners resolve this using consensus mechanism

Consensus mechanism impacts TX finality

- At the start of each round, miners build a block and propagate that to the rest of the network as soon as possible.
- In the meantime, there's a possibility other miners may also build a block with a different combination of transactions.
- In PoW, even for the same set of transactions included in the block, there can be different solutions for the PoW puzzle.
- Even when a miner builds a block and propagates that to the network, it takes time to propagate that block to most of the nodes in a network due to network delay. In the meantime, other miners will continue to build blocks.
- So, we need a mechanism to resolve which block to be accepted as included in the chain of blocks among these competing blocks for the same block number.
- This is achieved using a consensus mechanism.
- The chosen consensus mechanism has an impact on the TX finality.

2 Main Families of Consensus Mechanisms

Nakamoto Consensus

New block/TX is final when it's on the "biggest" chain we have seen so far

Probabilistic finality

- Can get orphans/uncles

Don't need to know how many nodes are there (> 1,000 ok)

Supermajority

New block/TX is final when I have seen evidence that > 2/3 of nodes agree

Conventional finality

- Like normal database TXs

Need to know how many nodes there are (~10–100 nodes ok)

- May know there are different parties

- Consensus mechanisms/protocols can be broadly classified into 2 families.

We have already seen the Nakamoto consensus.

A new block and its transactions are final when it's on the "biggest" chain we have seen so far. The biggest is in terms of computing power.. The term "Biggest" is more appropriate than "longest", as blockchains like Ethereum include uncle blocks (predecessor blocks that are not in the longest chain) to increase the weight of a block, making a block carry more computation behind.

Whether a block is included in the longest chain or not depends on the probability of an alternative longer chain appearing. With time, this probability goes down as more and more blocks are added to the current longest chain. When a TX can't be reverted, we consider it to be final. Hence, TX finality is probabilistic in Nakamoto consensus-based blockchains. A typical assumption in Bitcoin is to wait for 6 blocks along the same chain of blocks after a TX is included in a block, as it is probabilistically quite unlikely to be no longer in the longest chain (probability is never zero). Likewise, Ethereum 1.0 used wait for 10-12 blocks. However, there are cases where waiting for a few blocks is not enough if you need a very high-level of assurance. E.g., In 2013, Bitcoin forked due to a bug in one version of the software but not another version, leading to part of the network rejecting a chain that was accepted as dominant by the other part. The split was resolved after 6 hours. Also, due to an attack in 2010, ½ day worth of blocks were reverted. Ethereum DAO attack is another example. If a block is no longer in the longest chain, we call it an orphan block (usually in Bitcoin). Such blocks in Ethereum that are later referred to by a successor block are called uncles. These protocols don't need to know how many nodes are there, and 1,000s of nodes are common. More nodes mean more computing power, which is handled through automatic block difficulty adjustment based on inter-block time. If the inter-block time of the last few blocks is low, the difficulty is increased. If inter-block time increases, the difficulty is reduced.

- Other family of protocols rely on some form of supermajority votes.

New block/TX is final when I have seen evidence that > 2/3 of nodes in the network agree. This achieves finality in the conventional sense where a transaction is final as soon as it's included in a block that achieves super majority, e.g., like normal database transactions. However, to determine the supermajority, we need to know how many nodes there are in the network (these networks typically have 10 to a couple of 100s of nodes). These nodes may also know there are different parties in the network. Knowing exactly who is in a distributed network is hard. Hence, generally, network is static and nodes know at least the total no of nodes is supposed to be in the network. The supermajority used for voting could be a subset of all known nodes. For e.g., in Ethereum 2.0, for each epoch, 128 nodes are randomly picked to attest on a block.

- Side note – Finality

After a transaction is complete if the stated asset owner (e.g., payment recipient) can claim it's legally theirs we consider the transaction to be "final". In information systems point of view, this means once the transaction is recorded it will not change. However, there is no system in the world that offers truly 100% settlement finality in the literal sense of the term. Even on a paper-based asset registry, one can change an entry by flipping letters. This is much easier on a computer system with one or a few replicas. Also, acts of god can lead to the loss of physical and electronic records...

Proof of Work (PoW)



Nodes compete for the right to create next block by solving a hash puzzle

- Takes effectively random time
- *Prob (winning next block) = Fraction of global hash power miner controls*

Energy intensive, high latency

- Electricity consumption of Bitcoin is compared to countries/cities, e.g., Turkmenistan & Las Vegas
- Probabilistic finality



Put PoW for good use

- Primecoin generates prime number chains which are of interest to mathematical research

- As we have discussed so far, PoW miners compete for the right to create a block by solving a hash puzzle, e.g., Bitcoin and Ethereum 1.0.
- To create a block, a miner needs to find a nonce that makes the hash of the concatenation of the strings (nonce plus block header data) meet a certain difficulty level, e.g., a given no of leading zeros.
- Hashing functions are uni-directional, making it extremely difficult to solve, and easy to verify a given answer
- Hence, the only way to find a nonce that solves the hash puzzle is to try a large enough number of nonces until you get lucky; hence, it takes a random time effectively.
- The more computing power you throw into solving the puzzle by trying out multiple nonce values at the same time (i.e., parallelly) determines your chance of solving the puzzle.
- Therefore, the probability of finding a matching nonce is proportional to the fraction of the global hash power the miner controls.
- Given the massive amount of computing resources you need to solve the hash puzzle, this process is extremely energy intensive.
 - For e.g., the electricity consumption of Bitcoin is compared to countries like Turkmenistan or cities like Las Vegas.
- Due to probabilistic finality, you must wait for some time to be sure that the TX is included in the biggest chain with high probability.
- Blockchains like Primecoin try to put the work done in solving PoW puzzle to good use. Primecoin generates prime number chains which are of interest to mathematical research.

PoW Cont.

Answer is difficult to compute, but easy to validate

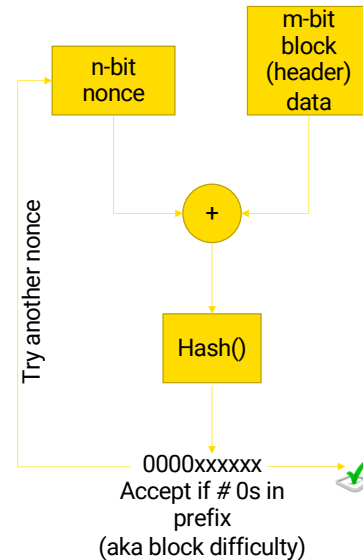
- No shortcuts, must try all possible answers

Difficulty of problem is automatically adjusted with time

- Overcomes increasing computing power to maintain average inter-block time

Miners use specialised hardware to try multiple nonces at once

- ASICs in Bitcoin & GPUs in Ethereum



WK 04-02

20



- We have already seen this slide.
- Once a set of pending transactions are included in a block, miners need to solve a puzzle to build a block. Once they solve the puzzle only they can announce/propagate/broadcast the new block into the network.
- This puzzle is called Proof of Work (PoW) as it is difficult (computationally expensive, costly, and time-consuming) to solve the puzzle but easy for others to verify the answer. This is more like you solve a problem by spending an hour, while your teacher can verify your answer in seconds.
 - Bitcoin miners solve a puzzle called hash cash to build a valid block. Ethereum used an algorithm called Ethash (Ethereum 2.0 doesn't use PoW).
 - PoW algorithms like hashcash are “embracingly parallel problems” with no shortcuts to finding the answer.
- The diagram outlines the basic idea:
 - On one side you have the data that reflect the content of the block header like Merkle root, previous hash, and timestamp.
 - The bitcoin block header is 80 bytes including a nonce, i.e., $m = 80 - 4 = 76$ bytes.
 - On the other side, you have the nonce, a large random number.
 - In Bitcoin nonce is 32-bits (i.e., $n = 4$ bytes) and the resulting hash value is 256 bits.
 - You concatenate the 2 and calculate the hash value.
 - You then check whether the resulting hash value satisfies a certain property. E.g., For here we check whether

the hash has 4 zeros as the prefix. Such a hash is called a valid hash and the associated block is called a valid block.

- In practice, the acceptance threshold can be more specific like a very large number.
- The acceptance threshold is adjusted over time to ensure the average inter-block time remains 10-min with increasing computing power.
- If not, you must retry again while changing the nonce. You can't change the m-bit data, as it reflects the content of the block. If you want to do that you have to do more work than just trying another nonce.
 - There's no easy way to guess what nonce would work. So you must try nonce values until you get lucky.
 - Hence, quite a massive number of hashes need to be tried before finding a hash value that satisfies the acceptance threshold. Thus, the process is computationally expensive and consumes lots of power.
- While it's difficult to find what nonce works, easy to validate whether a given nonce satisfies the condition.
 - Just concatenate the claimed nonce with the block header, calculate the hash, and check whether the hash value satisfies the acceptance threshold.
- The difficulty of the problem is automatically adjusted with time to overcome increasing computing power to maintain average inter-block time.
 - A simple adjustment would be to increase/decrease the number of leading zeros in the hash. But a finer control can be achieved by adjusting the massive number that the hash value is expected to satisfy.
 - Bitcoin – Every 2,016 blocks (approximately 14 days given roughly 10 minutes per block), nodes deterministically adjust the difficulty target/threshold.
 - Ethereum – Difficulty is adjusted every block.
 - This essentially means more computation the BC network put into solving the puzzle, and the difficulty of the puzzle gets increased, increasing the time to solve the puzzle. That way we can retain an average ~10-min inter-block time target.
- Miners use specialised parallel hardware to speed up the process by computing multiple nonce values parallelly:
 - ASIC – Application-specific integrated circuit
 - GPU – Graphics processing units
 - As of April 2022, it takes on average 122 sextillion (122 thousand billion billion) attempts to generate a block hash smaller than the difficulty target.
- The actual mechanism may slightly change from the given figure, e.g., instead of changing nonce on the block, miners may add a random value to coinbase TX to keep changing the block header until block difficulty is achieved. See <https://en.bitcoin.it/wiki/Mining>

Proof of Stake (PoS)



Select next mining node based on the control of native cryptocurrency

- Align incentives of cryptocurrency holders with good operation
- Add some randomness to prevent selecting only miners with the largest stake
- More cost-efficient, low latency

Delegated Proof-of-Stake

- Accounts delegate their stake to other accounts rather than participating in the TX validation
- Bitshare
 - Representatives take turns in a round-robin manner to build blocks



- In proof of stake (PoS), miners (mostly called validators) are selected in proportion to their quantity of holdings in the native cryptocurrency.
- A validator with a higher stake has a high chance of building a block. This idea is that such validators have stronger incentives to make sure the network operates as expected. Otherwise, their assets are at risk of losing value.
- Some randomness is added to the selection of validators to prevent selecting only miners with the largest stake.
- PoS is more cost-efficient and has low latency. However, it's considered to be not as secure as PoW.
- Peercoin was the 1st blockchain to introduce PoS
- It's used in many other blockchains such as Cardano, Algorand, Avalanche, Polkadot, Tron, and EOS.
- Delegated Proof-of-Stake (DPoS) is a sub-variant where account delegate their stake to other accounts rather than participating in the transaction validation.
- DPoS is considered to be less secure due to the centralisation of validators, as only a few validators act on behalf of others.
- Bitshare is a DPoS blockchain where representatives take turns in a round-robin manner to build a block.

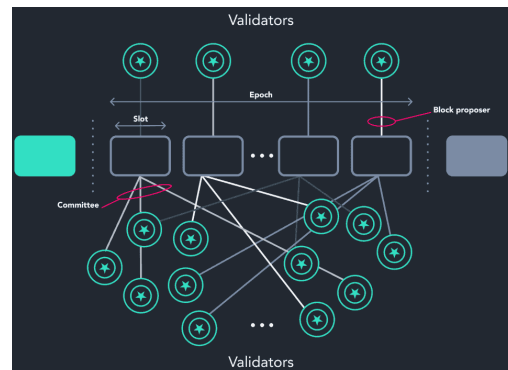
PoS – Ethereum 2.0

Nodes become validators by staking 32 ETH

- Stake is reduced if no show when called upon
- Stake is slashed if build an invalid block

12 sec slots, 32 slots for epoch

- A randomly selected validator proposes a block for the slot
- A committee of validators is also selected randomly & votes for the proposed block
- All validators vote for pairs of checkpoints
 - 1st block in epoch is a checkpoint
- If a pair of checkpoints attracts votes representing $\geq 2/3$ total ETH staked, checkpoints are upgraded – one CP justified and other finalised



Source: <https://www.bloxstaking.com/documents/eth2/>

- PoS in Ethereum 2.0 is quite complicated. Here we take a very simplified view, see <https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/> for some more details.
- A node becomes a validator (miners in Ethereum 2 are called validators) by staking 32 ETH to a smart contract.
 - Validator will lose part of the stake if no show when called upon to build a block, i.e., being off-line when they are supposed to propose a block.
 - The stake is slashed if the validator builds an invalid block as determined by the majority of other validators. Some consider this to be like burning down one's PoW computing farm/warehouse.
- Time in proof-of-stake Ethereum is divided into slots (12 seconds) and epochs (32 slots).
- One validator is randomly selected to be a block proposer in every slot.
- A committee of validators (128) is also selected randomly and votes on the proposed block. A block is valid if the majority of them agree on the proposed block's validity.
- All validators vote for pairs of checkpoints (1st block in the epoch is called a *checkpoint*).
- If a pair of checkpoints attracts votes representing $\geq 2/3$ total ETH staked, checkpoints are upgraded.
 - The more recent of the two (target) becomes "justified". The earlier of the two is already justified because it was the "target" in the previous epoch. Now it is upgraded to "finalized".
- Thus, you need to wait for 2 epochs to achieve finality in Ethereum 2. Thus, the time to finality in Ethereum 2 is higher than Ethereum 1.

Proof of Authority (PoA)



Known nodes are assigned block building role

- Attached reputation of an identity incentivises good operation
- Ensures consensus despite arbitrary behaviours from a fraction of participants

More conventional approach within distributed systems

- Validators may work as a group or elect a leader
- Small number of validators → Centralisation

High throughput & lower latency

Authority becomes a target for attackers

Used in permissioned blockchains

- A few IOs of nodes max
- All participants agree on who will be in the network

WK 04-02

23



- In PoA-based networks, transactions and blocks are validated by approved/known accounts, known as validators.
- With PoA, individuals earn the right to become validators. Hence, there is an incentive to retain the position that they have gained. By attaching a reputation to identity, validators are incentivised to uphold the transaction process, as they do not wish to have their identities attached to a negative reputation.
 - It can be seen as a consensus model with identity at stake
- This is a more conventional approach within distributed systems where the validator role may be upheld by all nodes or a leader may dynamically elected to validate transactions and blocks on behalf of other validators.
- However, the network has only a small number of validators making the consensus process centralised.
- PoA provides high throughput and lower latency as TX finality is immediate and small size of the network.
- However, there's a high risk of authority getting compromised as it's an easy target for attackers.
- Therefore, PoA is typically used in permissioned blockchains and validators are well-guarded.
- All participants agree on who will be in the network, e.g., to add a new order to an existing order network in Hyperledger Fabric, the existing order should approve that TX.
 - Hyperledger Fabric is moving toward BFT-based consensus (see next slide).
 - Another PoA DLT is R3 Corda.
- You can also run existing public blockchains under PoA consensus, e.g., Quorum and Hyperledger Besu are based on Ethereum and can use PoA consensus.

Practical Byzantine Fault Tolerance (PBFT)

Algorithms based on Byzantine Fault Tolerant (BFT)

Use a known set of block validators

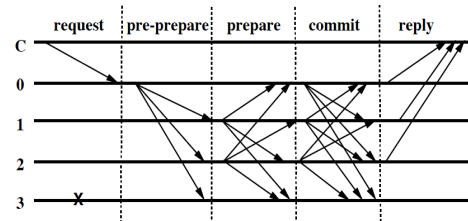
- New block is created every x seconds
- Each block must be confirmed by 2/3 majority

High throughput & lower latency

Used in permissioned blockchains

- Use $3f + 1$ nodes to guard against f failures
- All participants agree on who will be in the network
- 10s to a few 100 nodes

Known attack if some validators misbehave



- Byzantine Fault Tolerance(BFT) is the feature of a distributed network to reach consensus even when some of the nodes in the network fail to respond or respond with incorrect information (i.e., either fail or misbehave).
- The objective of a BFT mechanism is to safeguard against system failures by employing collective decision-making (both – correct and faulty nodes), which aims to reduce to the influence of the faulty nodes.
- Practical Byzantine Fault Tolerance (PBFT) protocols are a subclass of BFT protocols that work in asynchronous networks, i.e., with no assumptions about network delay (whereas BFT protocols have strong assumptions about network delay and do not work across the Internet).
- PBFT has been applied for consensus in permissioned blockchains, e.g., Stellar, Tendermint, VMware Concord, and RedBelly.
- They use a known set of block validators/nodes to build and confirm blocks.
- A new block is created every x seconds (or no block is emitted if there are no transactions), and each block must be confirmed by a 2/3 majority.
- As we can see from the figure:
 - Once a TX is submitted by a client (C), it goes into one of the nodes (called primary).
 - The primary then broadcasts the transaction to all other validators (called replicas) – called the pre-prepare step.
 - A validator validates the transaction and replies its valid/invalid decision to the primary node – Prepare step.
 - Once a sufficient no of positive votes is received (usually 2/3 majority), primary broadcast all other validates to commit the transaction – commit step.
 - Then only all nodes update the ledger.
 - The client is notified by each validator so that the client can verify a majority of the network executed the TX and update the ledger state.
 - Then for the next round, a different validator is selected to the primary. Or this may happen by getting clients to send transactions to any of the validators.
 - This idea can be extended to a block, where the primary first collects all TX and starts the pre-prepare step.
- PBFT solutions usually have high throughput & lower latency as the transaction is final after the commit step. These properties may not be as high as PoA consensus, but network size tends to be relatively large and can spread across the Internet.
- PBFT is usually used in permissioned blockchains.
 - Usually uses $3f + 1$ nodes to guard against f failures. If ' f ' nodes are byzantine failures, then the system needs to deal with two types of problems. First is not sending the message at all and second is maliciously sending a different message. So the system needs to function well after ' $(n-f)$ ' nodes. However, it is possible that the ' f ' replicas that did not respond are not faulty and, therefore, the ' f ' replicas that responded might be faulty. So, if we want the non-faulty nodes to outnumber the faulty ones; we need at least ' $(n-f)-f > f$ '. Therefore, ' $n > 3f+1$ ' is optimal.
 - All participants agree on who will be in the network.
 - 10s to a few 100 nodes can be in the network and they may be geographically distributed.
- There are known attacks for PBFT where some validators misbehave or their behaviour could substantially slow down the network.

Other Alternatives

Proof of Retrievability (PoR)

- In proportion to distributed storage of archival data

Proof of Elapsed (PoE) time

- Wait a random time to create the next block
- Using Trusted Execution Environment (TEE), e.g., Intel SGX
 - Run trusted code in trusted environment
 - Ensure the wait times are created fairly

Proof of X (PoX)

- A generic term to indicate use of some form of proof

- There are many other consensus designs with varying properties.
- Proof-of-retrievability is based on the ability to retrieve data spread across a distributed system. More data you hold you have a higher chance of building a block.
- Proof-of-Elapsed time is based on the idea that the next block is generated after waiting for some random time.
 - Use specialised hardware like Intel SGX to prove that the node waited for a random time based on a chosen algorithm before emitting the block. Intel SGX allows running trusted code in a trusted environment.
 - Ensure the wait times are created fairly.
- PoX is a generic term used to indicate the use of some form of proof.

Agenda

Taxonomy definition

Blockchain taxonomy

- (De)centralization
- Consensus protocols
- Ledger structures
- Mining & ledger distributions

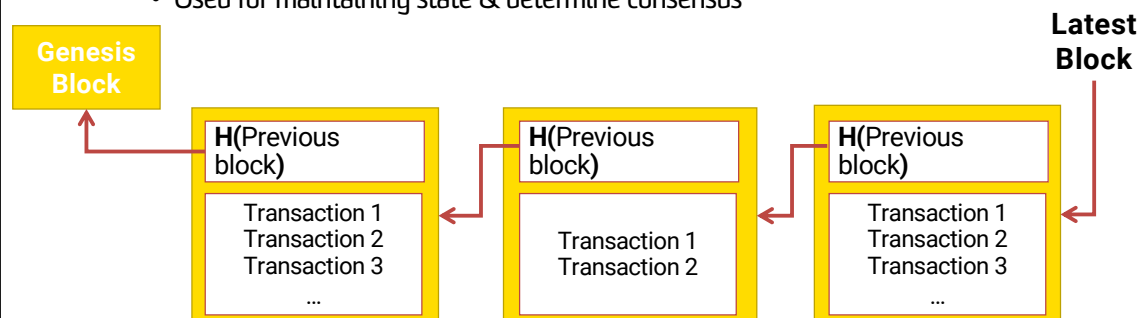
List

Blockchain appears as a logical list of blocks to a user

- E.g., Bitcoin & Ethereum

Nodes record blockchain as global tree of blocks

- Shorter branches attached to main chain represent alternative competing histories
- Used for maintaining state & determine consensus



- So far, we discussed blockchain as a (linked) list of blocks, where the history of all transactions is captured in the blockchain structure.
- While Ethereum 2.0 follows this list structure, Ethereum 1.0 and Bitcoin clients actually record the blockchain as a tree of blocks, where the shorter branches attached to the main chain represent alternative competing histories (i.e., chain forks).
- This structure is used to maintain the state and determine consensus. In many cases, while the chain of blocks appears as a logical structure, the actual data structure that persist data is different, e.g., some blockchains use a key-value store.

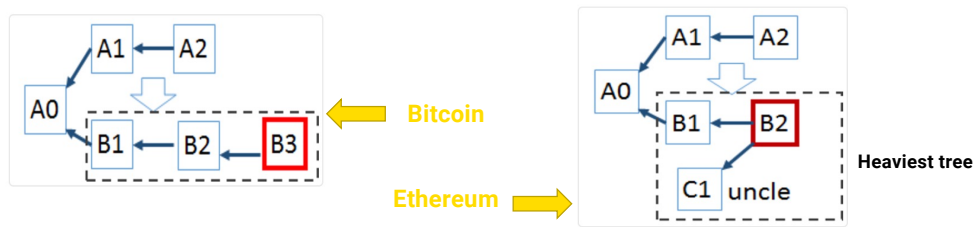
Tree

Orphan/stale (uncles) blocks

- 2 nodes find a block at the same time
- Propagated, verified, but eventually being cast off
- Low inter-block time suffer from a high number of stale blocks

Ghost (Greedy Heaviest-Observed Sub-Tree)

- Add stale blocks into calculation of heaviest chain



WK 04-02

28

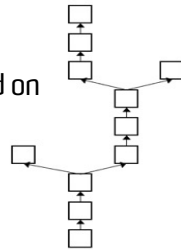


- An orphan or stale block, is created when 2 nodes find a block at the same time. This results in a tree of blocks.
- Both found the solution to a block and send off their block to be verified and included in the blockchain. Usually completing blocks have different combinations of transactions.
- But one of them eventually gets cast off as a longer chain achieved dominance.
- In Bitcoin, the probability of finding a block at the same time is relatively low as the inter-block time is 10 minutes and propagating a block to 50% of the network takes roughly 12 seconds.
- Whereas in Ethereum 1.0 miners are likely to make multiple competing blocks.
- Therefore, Ethereum 1.0 used Ghost (Greedy Heaviest-Observed Sub-Tree) protocols to reference competing independently-mined blocks (aka *uncle* blocks), to add weight to their chain for its selection as the main chain.
- This recognition of concurrent work allows shorter inter-block times which can improve the overall throughput of the blockchain system.

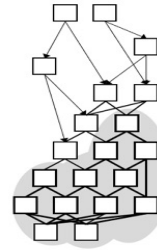
Directed Acyclic Graph (DAG) of Blocks/TXs

Global DAG

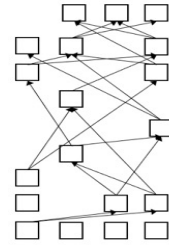
- Logical view of transactions is based on a DAG of TXs or blocks
- TXs – Tangle in IOTA
- Blocks – Hashgraph in Hedera



Blockchain



Tangle



Hashgraph

	Bitcoin	IoT	Hedera
Byzantine Tolerance	51%	34%	34%
Confirmation Time	60 min (6-Confirmation)	Unstable	5 sec

- The third option is to maintain a global Directed Acyclic Graph (DAG) of blocks or transactions.
- The first figure shows the tree view of a blockchain (the most recent block at the top).
- 2nd figure is from IOTA, a platform that maintains a DAG of transactions rather than blocks. The proposed data structure is called Tangle.
 - In IOTA, a TX performs PoW by verifying that 2 previous transactions are valid. Once that transaction is confirmed by another transaction it is considered to be final. This also means time to finalise a transaction largely depends on how active the network is.
 - IOTA does not have transaction fees as each TX has to spend resources to validate 2 previous transactions.
- 3rd figure is a DAG of blocks, where a block points to 1 or 2 more previous blocks. This structure is called hashgraph and was introduced by Hedera.
- Compared to Bitcoin, IOTA is more vulnerable, and can tolerate up to 34% power attacks (this is a theoretical attack that requires large computing power and grates visible on the major fraction of the DAG).
- The confirmation time on Bitcoin is 60 minutes, the confirmation time on IoT is unstable and mainly depends on how active the network is.

Peer-to-Peer Ledger

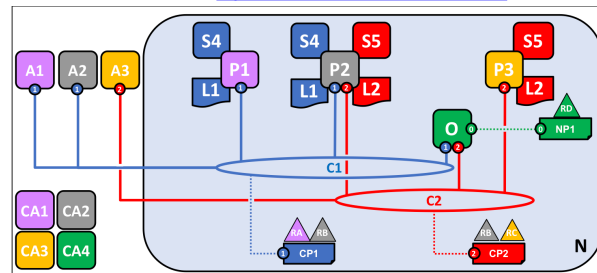
Hyperledger Fabric

- A collection of small ledgers, aka channels
- Isolate transactions within a channel

R3 Corda

- Transactions are only distributed to parties of interest
 - Parties see a collection of small ledgers, shared with other business contacts
 - Abstract logic view is a global graph of transactions
- Notaries are used to further limit transaction distribution
 - Attest to integrity of unseen parts of transaction graph

Source: [Hyperledger Fabric Network](#)



P1, P2, & P3 – Clients
 RA, RB, RC, RD – Organisations
 C1 & C2 – Channels
 CA1 – CA4 – Certificate authorities
 L1 & L2 – Ledgers
 S4 & S5 – Chain code (Smart contracts)
 A1, A2, A3 – Applications
 O – Ordering service
 RARB & RBRC – Consortia
 CP1 & CP2 – Channel policies

- Hyperledger Fabric (HLF) maintains a collection of small ledgers, called channels, which can apply a more rigid transaction distribution policy that isolates transactions within the channels.
- Transactions within a channel are visible to its members but not to others.
- The diagram gives a simplified architecture of an HLF deployment with 4 organisations, each represented by a different colour.
 - Organization 1 and Organization 2 need private communications within the overall network through C1, as do Organization 2 and Organization 3 through C2.
 - Peer node P1 maintains a copy of the ledger L1 associated with C1.
 - Peer node P2 maintains a copy of the ledger L1 associated with C1 and a copy of ledger L2 associated with C2.
 - Peer node P3 maintains a copy of the ledger L2 associated with C2.
 - Channel C1 is governed according to the policy rules specified in channel configuration CP1; the channel is under the control of organizations RA and RB.
 - Channel C2 is governed according to the policy rules specified in channel configuration CP2; the channel is under the control of organizations RB and RC.
 - There is an ordering service O4 that serves as a network administration point for N and uses the system channel. Administrative rights to organization RD.
- In R3 Corda, transactions are only distributed to parties of interest.
 - Most parties see a collection of small ledgers, each shared with their related business contacts involved in a transaction. Essentially, if 2 parties ever transacted once, it's like having a ledger with only those 2 parties and only that transaction.
 - The abstract logical view of transaction history is a global graph of transactions.
 - Special agents (notaries) can be used to further limit the distribution of transactions while attesting to the integrity of unseen parts of the transaction graph.

Ledger Structures Trade-Off Analysis

Option	Fundamental properties	Impact			
		Cost efficiency	Performance	Flexibility	
Global list of blocks (Bitcoin)	⊕⊕⊕	⊕	⊕	⊕	Single global TX history
Global DAG of blocks (Hashgraph)	⊕⊕	⊕⊕	⊕⊕	⊕⊕	
Global DAG of transactions (IOTA)	⊕⊕	⊕⊕	⊕⊕	⊕⊕	
Restricted shared ledgers (Corda)	⊕	⊕⊕⊕	⊕⊕⊕	⊕⊕⊕	Multiple small ledgers

- Here is a summary of trade-offs of different ledger structures.
- A global list of blocks has most of the fundamental properties of blockchains (e.g., immutability and transparency), however, they come at the expense of performance, flexibility, and cost.
- Whereas HLF and Corda's approaches have better performance, flexibility, and cost efficiency, they compromise on fundamental properties.
- DAGs sit in the middle.

Agenda

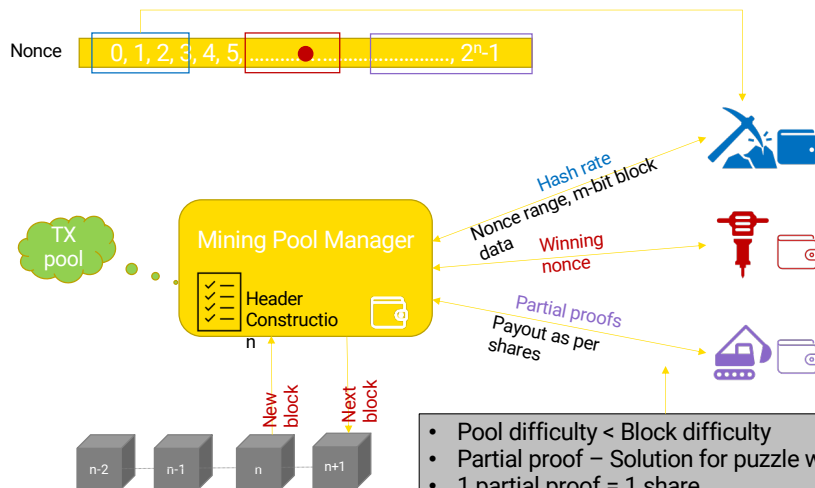
Taxonomy definition

Blockchain taxonomy

- (De)centralization
- Consensus protocols
- Ledger structures
- Mining & ledger distributions

- Next, we talk about a few options that optimise or reuse mining or distribute ledger structures to gain enhanced performance and cost efficiency

Mining Pools



WK 04-02

- Given the massive amount of computing power needed to solve PoW, solo mining is unlikely to work. Therefore, miners pool their computing power to form a massive virtual computer.
- A mining pool has a manager that it's typically controlled by one or more parties.
- As discussed earlier, as soon as a new block is received, the mining pool manager starts building the next block by including transactions from the transaction pool.
- We can think of nonce as a large set of numbers. If the nonce is n-bit long there are 2ⁿ nonces from 0 to 2ⁿ-1. E.g., if n = 8 there are 256 nonces from 0 to 255.
- As the nonce is very large it needs to be split and assigned to different miners. Only one of those nonces (or a few of them to be more accurate) will be the answer to the PoW puzzle.
- As miners bring in disproportionate mining power, the nonce range needs to be split based on their computing capacity.
 - This is usually done based on an estimate of the number of hash calculations that a miner can perform for a unit of time. This is called the "hash rate".
 - Higher the hash rate, the larger the nonce range allocation.
- A range of nonce values is then assigned to the miner. Similarly, other miners assigned their quota.
- Suppose once the miners are successful in solving the puzzle. It reports that back to the pool manager.
- The pool manager assembles the next block and propagates that to the rest of the network.
- Mining reward and TX fees are assigned to the mining pool owner's address. Which later transfers the payout to miners based on their contribution measured based on their "share" of contribution.
 - Share is measured based on the number and difficulty of partial proofs a miner submits.
 - Partial proofs (aka weak proofs) are answers to PoW puzzle with a slightly low difficulty level. E.g., if network difficulty requires you to find 16 leading zero bits, partial proof may require finding only 10 zero bits.
 - Miners are encouraged to submit multiple partial proofs to increase their share and show that they are mining.
- Actual mining pools may behave slightly differently from this diagram. But the basic mechanism remains the same.
- This idea can be extended to Proof of Stake blockchains, where miners pool their stakes.

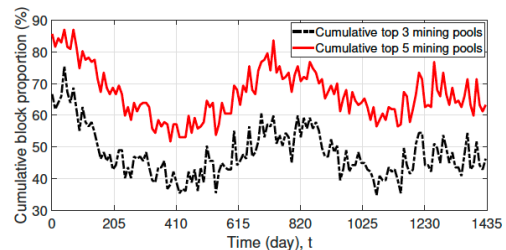
Mining Pools – Cont.

Mining pools aggregate computational power

- Better chance of solving PoW puzzle
- Receive more consistent block rewards & TX fees to pool's address
- Distributed to pool members' addresses as per their share

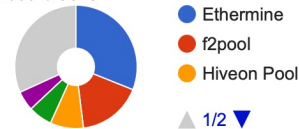
Pool manager sets business rules

- Payout is usually subject to a fee
- A few ways to game the system
- Mining isn't quite centralised, but pool decision making is



Source: C. Wang, X. Chu, and Y. Qin, Measurement and Analysis of the Bitcoin Networks: A View from Mining Pools, BigCom, 2020.

Miners of last 1000 blocks



WK 04-02

34



- Given high block difficulty solo mining may not work at all unless they were lucky in finding a nonce without much effort. But long-term this won't work.
- Mining pools aggregate computational power enhancing the chance of solving the PoW puzzle as a collective.
- While even a mining pool may not be able to build a set of adjacent blocks, it may be able to build one in several blocks.
- This increases the consistency of receiving a block reward and transaction fees.
- As every pool member gets some reward as per their share, everyone receives a more consistent reward.
- Most pools are managed by a few parties.
- These parties usually charge a fee, e.g., 3-5% from the payout.
- There are opportunities to game the system in terms of getting disproportionate shares or not paying certain miners.
 - An example attack is pool manager may not payout some miners.
 - One solution is for miners to share each other share information to double-check whether the pool manages payout properly, e.g., see https://en.bitcoin.it/wiki/P2Pool#Protocol_description
 - A bit more complicated issue is when the miner may have found the solution, but may try to find more partial proofs to increase the share before returning the correct proof to the pool. But miners risk the pool not being able to build the block before other pools.
 - Miners may find partial proof and stop further processing or jump to another pool. Miner gets some reward if another pool member gets the proof.
 - There are ways to mitigate some of these problems.
- As we can see from the 2 figures, a few mining pools are responsible for most of the computing power in Bitcoin (top diagram) and Ethereum (bottom diagram from ethviewer.live)
 - While mining is still decentralised as computing power comes from many nodes, mining pool decision-making is quite centralised
 - This has implications for the security of the consensus process and collusion of a few mining pool manager may result in a 51% attack.

Merged Mining

Bootstrapping problem for new blockchains

- Hard to attract enough computing power to maintain high block difficulty

For established blockchains, miners leaving for new ones is a problem

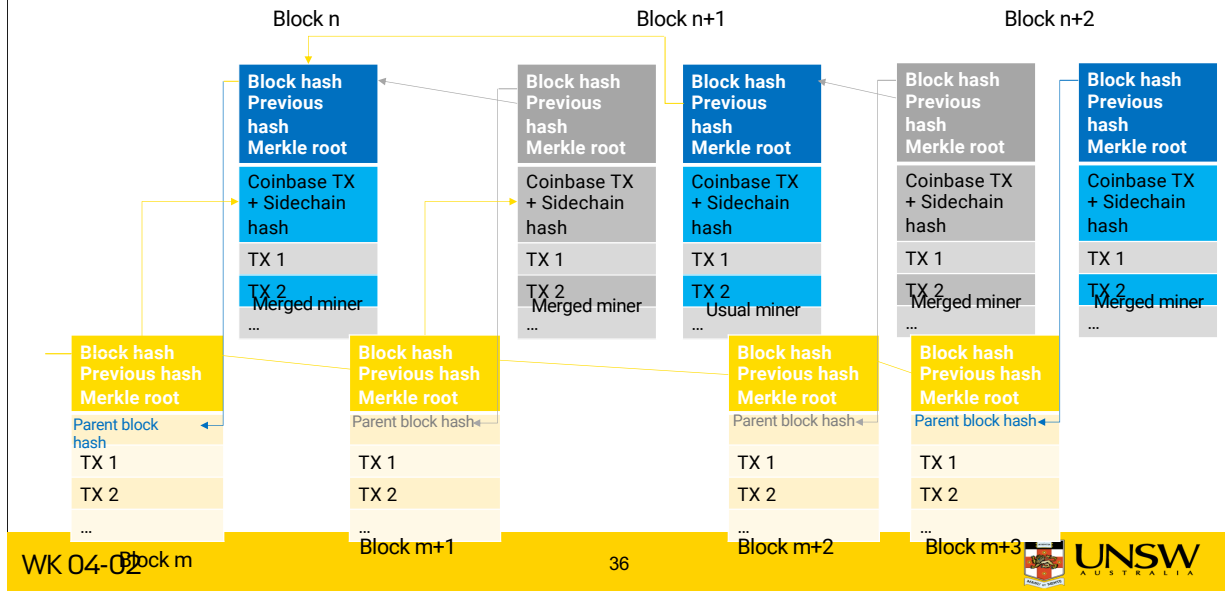
- Fragmented computing power due to miner hopping or leaving for good

Merged mining solves both problems, while increasing miners' income

1. Set sidechain's block difficulty < Parent chain's block difficulty
2. Build sidechain's next block header
3. Include sidechain's block header in parent chain as a TX (usually in coinbase TX)
4. Solve PoW puzzle on parent chain as usual
5. Accept a partial proof as answer to sidechain's puzzle
6. Propagate next block on sidechain with partial proof & parent chain's hash in header

- New blockchains are not able to accumulate a sufficiently large number of miners/computing power to have a high difficulty level to ensure their blockchain remains secure. This is called the bootstrapping problem.
- Alternatively, the computing power of a well-established blockchain gets fragmented when miners hop between blockchains or leave for good.
- Merged mining disincentive miners of large and established cryptocurrencies like Bitcoin to switch their mining activities to emerging cryptocurrencies where miners are given the possibility to continue mining on the established cryptocurrencies, while also generating profits in merge-mined child blockchains.
 - Merged mining – is the process of reusing partial/weak PoW solutions from a parent cryptocurrency as valid PoW for one or more child cryptocurrencies.
 - For new blockchains, merged mining provides access to the large computational capacities of established blockchains like Bitcoin.
 - A miner may merge mine for multiple child blockchains called “sidechains”.
 - Namecoin is the first blockchain that uses merged mining with Bitcoin.
- For a parent cryptocurrency to allow merged mining it must be possible to include arbitrary data in a transaction.
 - A single parent blockchain can be used to perform merged mining on multiple sidechains by including the Merkle-tree root of a Merkle tree of the sidechain's block hash.
- At a high-level merged mining process include the following steps:
 1. The first set sidechain's block difficulty is a bit less than the Parent chain's block difficulty.
 2. Build the sidechain's next block header by aggregating transactions and calculating Merkle root.
 3. Next, include the sidechain's block header in the parent chain as a TX. This is usually included in coinbase TX as that confirms the miner.
 4. Solve the PoW puzzle on the parent chain as usual.
 5. Accept a partial/weak proof that satisfies the sidechain's difficulty as an answer to the sidechain's puzzle (the parent chain's answer is also an answer for the sidechain).
 6. Finally, propagate the next block on the sidechain with its block data, block header which now contains partial proof and the parent chain's hash in the header.
- A good description of merged mining is given at <https://alexeizamyatin.medium.com/a-technical-deep-dive-into-merged-mining-5b67706e1a19>

Merged Mining Example



- Here's a simplified illustration of how merged mining works.
- The first miner on the side chain builds a block based on the Merkle root of its set of transactions and the previous block's hash. It then calculates the block hash.
- The block hash is then included in the coinbase TX of the parent chain's merged miner (the same node may build a block for both parent chain and sidechains or it could be 2 different nodes).
- The parent blockchain miner then continues to solve the PoW puzzle as usual.
- If the miner is successful, it will propagate the block on the parent chain.
- At the same time, the parent chain's hash (and also the Merkle tree root of the parent block to confirm coinbased TX) is included in the sidechain's block.
- Then the sidechain's block is broadcasted to the sidechain. Anyone in the side chain can verify this block by checking with the parent chain.
- Next, we look at an example where the parent chain's merged miner is only able to solve the partial/weak puzzle.
- As the partial proof is still valid due to the lower block difficulty in the sidechain, this nonce, and associated block data can be used to build the next block on the sidechain.
- Here we assume a non-merged minder (i.e., a usual miner) was able to solve the puzzle for the parent chain. Hence, this information is not used to build the next block on the sidechain.
- When it comes to the next round, we consider 2 scenarios.
 - In the first case, a merged miner was able to quickly find a partial proof based on which block m+2 was propagated on the sidechain.
 - In the second case, when the merged miner who was successful in solving the puzzle for the parent chain started building its block, it used the hash from block m + 3. Once it solves the puzzle its hash can be used to propagate block m+3 on the side chain.
- Hence, it is possible to generate multiple blocks for a sidechain for a single block in the parent chain.

Merged Mining Pros & Cons

No change to parent chain protocol

- Merged miner includes a (coinbase) TX with hash of sidechain block
- Merge miner doesn't attempt to validate sidechain block
- Many mining pools merge-mine several sidechains, e.g., Ghash.IO: Bitcoin, Namecoin, IXCoin, Devcoin

Sidechain client

- Construct header for sidechain & validate other sidechain blocks
- Verify parent chain's full or partial proof based on header
- Append parent chain's hash & propagate sidechain block

A small miner (or mining pool) on a large parent chain can ruin a sidechain

- Increased centralisation issues in sidechain
- Miner may mine on different branches of the same sidechain
- E.g., CoiledCoin by Eligius pool (Jan 2012), TerraCoin by unknown (Jul 2013), WorldCoin by unknown (Nov 2013)

WK 04-02

37



- Merge mining offer several advantages, but can also ruin a sidechain
- From the parent chains point of view, it does not require any changes to the parent chain's protocol
 - Merged miner just includes a (coinbase) TX with hash of sidechain block, or Merkel root if supporting multiple sidechains
 - Merge miner doesn't attempt to validate sidechain block
 - Many mining pools merge-mine several sidechains, e.g., Ghash.IO: Bitcoin, Namecoin, IXCoin, Devcoin
- Sidechain client is a bit more complicated
 - It needs to construct header for sidechain and share that with the parent chain miner.
 - It also needs to validate other sidechain blocks
 - Verify parent chain's full or partial proof based on header (usually parent chain's transactions are not validated)
 - It then appends parent chain's hash to new block and propagate the sidechain block to rest of the sidechain
- However, things can go wrong as merged miner's (or a mining pool) computing power may be too much given lower difficulty of sidechain
 - This could be seen as increasing centralisation concerns in sidechain
 - There's also risk of a miner mining different branches of the sidechain. This way, miner can attack by preventing Nakamoto concerns or substantially delay it
 - There are several well-know examples of this going wrong
- One of the well-known story is about an old coin called CoiledCoin, which was quite small
 - In 2012, and one of the large mining pools, Eligius in Bitcoin decided that CoiledCoin was a scam, and a bad idea.
 - Eligius pool pointed its massive computing resources at CoiledCoin and launched an attack.
 - The attack involved mining a lot of blocks that reversed days of TX history in CoiledCoin, reversing the user transactions. And as well as mining a long chain that had empty blocks containing no transactions. This is essentially a denial-of-service (DoS) attack.
 - CoiledCoin users weren't able to make any transactions during that time.
 - After a fairly short time of the siege, all of the users of CoiledCoin had left and moved on, and the altcoin doesn't exist anymore as such.
 - Merge mining makes it cheaper for attackers since the attackers don't even have to stopping mining Bitcoin while it launches this attack.

Sharding

Full nodes require sizeable storage space

Sharding

- Divide ledger state into pieces
- Participating blockchain nodes only hold some shards
- Enable parallel transaction execution
- TX sharding vs. State sharding

ID	Asset
Alice	500
Bob	1000
Charlie	500
Dave	Plot 123 @ 2015
Sweet Mango	Bowen QUE, Org. Cert #

ID	Asset
Alice	500
Bob	1000
Dave	Plot 123 @ 2015
Sweet Mango	Bowen QUE, Org. Cert # 45781

ID	Asset
Alice	500
Charlie	500
Sweet Mango	Bowen QUE, Org. Cert # 45781
Bob	1000
Charlie	500
Dave	Plot 123 @ 2015

WK 04-02



UNSW AUSTRALIA

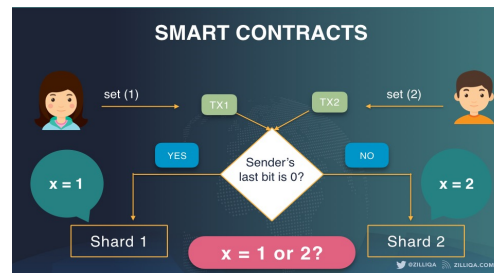
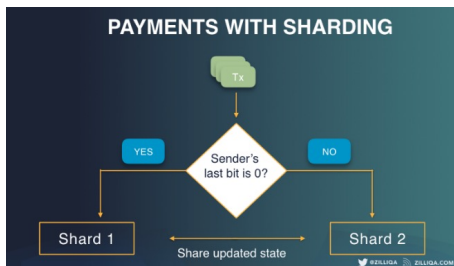
- The full nodes of most blockchain networks need to keep all historical transactions and the ledger state, which requires sizeable storage space.
- To reduce the storage burden of blockchain participants and address other scalability concerns, the concept of *sharding* from databases is also being applied to blockchains.
 - The idea is to split the ledger into different fragments and keep multiple copies of fragments on some number of replicas. Each fragment is called a *shard*.
 - Transactions on a shard can run parallel to transactions on other shards increasing transaction throughput.
- Sharding is quite complicated and nontrivial in blockchains. There are three types of sharding network, transaction, and state. We'll next discuss the latter 2.

Transaction Sharding

Shard to execute transaction is determined based on sender's address

Can prevent double spending

Doesn't work well with smart contracts



WK 04-02

39



- Zilliqa is a blockchain platform using sharing mechanism. Different transactions use different sharing strategies.
- Here we look at transaction sharding where transaction sender's address is used to determine which shards the transaction should apply.
 - To assign payment transactions in a network with two shards S1 and S2, we first check the sender's address. If the address ends with 0, then it is assigned to S1. Otherwise, it is assigned to S2.
 - As all ledger state of an address stays in the same shard, this can prevent double-spending.
- However, this doesn't work well with smart contracts .
- Let us now apply our transaction assignment strategy (based on the sender's address) for a transaction T1 that comes from Alice in which she calls set with an input 1. This transaction will go to say shard S1 and the shard will change the value of x to 1. So far, so good.
- Imagine another user Bob issuing another transaction T2 at the same time as Alice in which he calls set with an input 2. Again with 50% chance, T2 may end up being processed in S2. So, T1 gets processed in S1, while T2 gets processed in S2. In this case, S1 will set x to 1, while, S2 will set y to 2. This leads to an inconsistent contract state.
- Again, the two shards will have to communicate directly or via an intermediary to make sure that the two transactions get an ordering that will allow the contract's state to change in a deterministic manner and not remain in a "dirty" state.
- Details here - <https://blog.zilliqa.com/provisioning-sharding-for-smart-contracts-a-design-for-zilliqa-cd8d012ee735/>

State Sharding – Ethereum 2.0 (not in the roadmap)

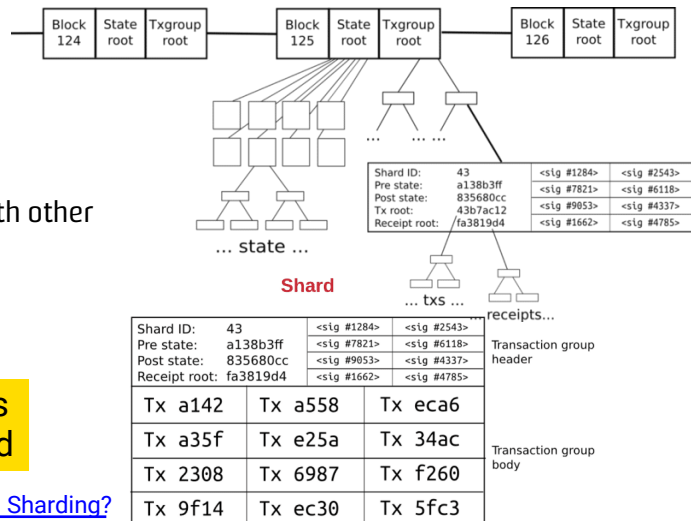
After sharding is activated

- Ledger state is split into shards
- Each account is in 1 shard
- Accounts can easily transact with other accounts within same shard

2 levels of interaction

Random validator who needs to verify transactions in shard

Source: [What are Ethereum Nodes And Sharding?](#)



WK 04-02

40



- Ethereum 2.0 was planning to improve scalability through sharding based on structuring the network into two layers. While details kept changing see <https://blockgeeks.com/guides/what-are-ethereum-nodes-and-sharding/> for a good description. However, it seems Ethereum has dropped the idea of sharing and now focusing on Layer 2 solutions.
- After sharding is activated, Ethereum 2.0 was supposed to use state sharding, i.e., sharding is based on ledger state than transaction sender's address.
- Imagine that Ethereum has been split into many islands. Each island can do its own thing. They were talking about at least 32 to 64 shards.
- Each of the island has its own unique features and everyone belonging on that island i.e. The accounts, can interact with each other AND they can freely indulge in all its features.
- If they want to contact with other islands, they will have to use some sort of protocol.
- Each shard has its own group of a transaction. The transaction group is divided into the transaction group header and the transaction group body.
- First level: Transaction Group
 - Shard ID: The ID of the shard that the transaction group belongs to.
 - Pre-state root: This the state of the root of shard 43 before the transactions were applied.
 - Post state root: This is the state of the root of shard 43 after the transactions are applied.
 - Receipt root: The receipt root after all the transactions in shard 43 are applied.
 - Right part: Full of random validators who need to verify the transactions in the shard. They are randomly chose.
- Top level: Second Level
 - The state root represents the entire state, and as we have seen before, the state is broken down into shards, which contain their own substates. The transaction group root contains all the transaction groups inside that particular block.
 - Accept transaction groups rather than transactions
 - Transaction group is valid if 1) pre-state root matches the shard root in the global state and 2) signatures in the transaction group are all validated
 - After transaction gets in, the global state root becomes the post-state root of that shard ID

State Sharding – Ethereum 2.0 (Cont.)

Cross-shard communication

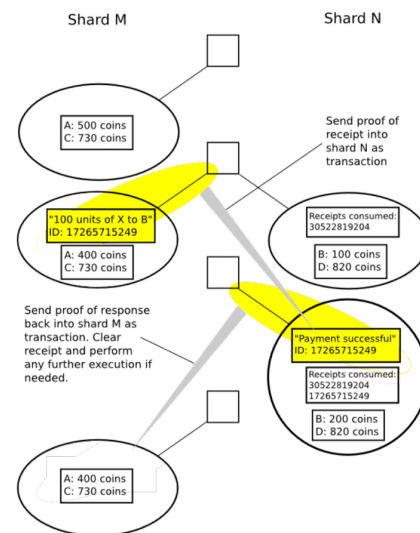
- Receipt of TX can be accessed via multiple Merkle trees from the TX group Merkle root

Transaction does

- Change the state of shard
- Generate a receipt

Receipts are stored in a distributed shared memory

- Seen by other shards, but not modified by them



WK 04-02

41



- Here's an example of a transaction that requires cross-shard communication.
- Suppose 100 tokens are transferred from account A to B, which sits on 2 shards.
- Transaction does change the state of the shard where account A resides (Shard M) and then generates a receipt.
 - Receipt of the transaction can be accessed via multiple Merkle trees from the transaction group Merkle root
- Receipts are stored in a distributed shared memory and can be seen by other shards, but not modified by them.
- Then seeing this receipt Shard N applies the other side of the transaction where the balance of B is updated.
- This also generates a receipt that is made visible to Shard N.
- This enables Shard M to finalise the transaction.
- Things are more complicated if the transaction fails on Shard N where Shard M needs to revert its changes.